

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Волгоградский государственный технический университет»

Факультет Электроники и вычислительной техники

Кафедра «ЭВМ и системы»

Контрольная работа

по дисциплине:

Мобильные и сетевые технологии

(указать название дисциплины)

на тему (№ варианта):

Создание простого веб-сервиса в Azure

(указать тему или номер варианта)

Выполнил(а)

студент(ка) группы

Ф.И.О.

ЭВМ-1.1

Иванова И.И.

Проверил(а):

Зав. кафедрой «ЭВМ», к.т.н., доцент

(должность, степень, звание)

Андреев А.Е.

(Ф.И.О. преподавателя)

Результат проверки:

(оценка, дата, подпись преподавателя)

Волгоград, 2020 г.

Содержание

Введение.....	3
1 Постановка задачи и описание RESTful интерфейса	3
2 Базовые классы сервиса.....	3
3 Создание и тестирование RESTful веб-сервиса на базе разработанных классов и размещение его в облаке	5
Заключение	11
Список использованных источников :.....	11

Введение

В данной работе рассматривается небольшое веб-приложение (веб-сервис) для работы со списком объектов из иерархии простых двумерных фигур. Для списка производится расчет общей площади и длины периметров фигур. Язык разработки - C#. Приложение работает с константным источником данных (который можно рассматривать как заглушку), а также с переданными в запросе параметрами. Используется платформа .NET / .NET Core.

1 Постановка задачи и описание RESTful интерфейса

Необходимо разработать небольшое приложение для обработки списка фигур с определением суммарной площади и длины периметра фигур в группе и разместить его в облаке Azure.

Таблица 1 – Описание REST интерфейса

Действие	Запрос

To be continued... ☺

2 Базовые классы сервиса

Базовая иерархия классов модели показана на рисунке 1 (диаграммы получены с помощью средства ObjectiF 7.1).

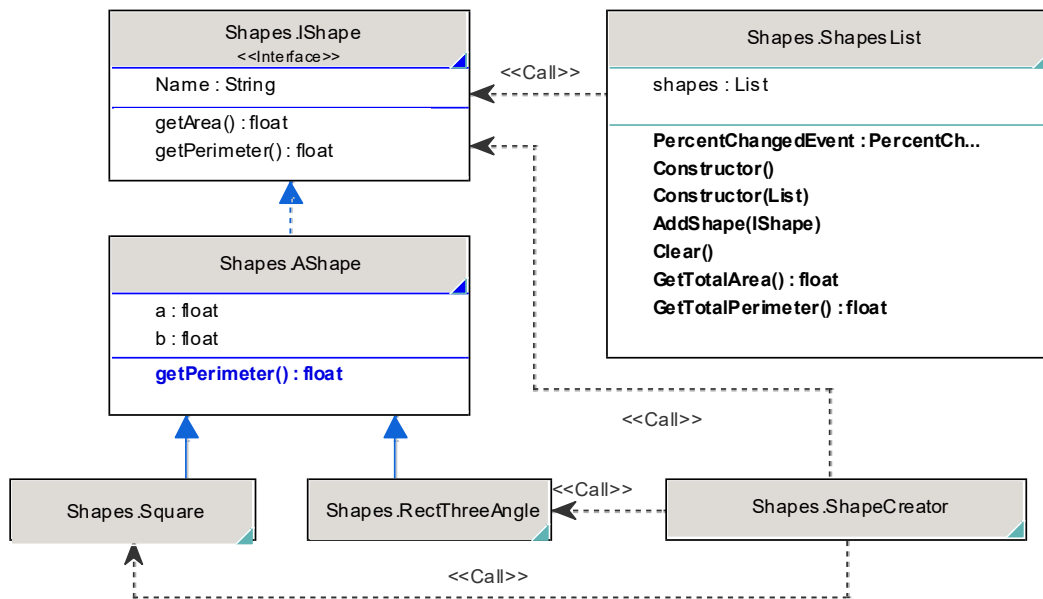


Рисунок 1 – Базовая иерархия классов фигур, списка фигур и создателя фигур

В этой иерархии выделяется интерфейс IShape, базовый класс с основными атрибутами (их 2 : a и b), конструктором, методом вывода в строку, а также две конкретных фигуры. Также реализован класс списка фигур ShapeList, собирающий статистику по фигурам (общая площадь и суммарная длина периметров), не зная об их типах. Соответственно это уже паттерн Полиморфизм (Стратегия), где стратегией является определение площади и периметра фигуры. Дополнительно реализован прототип фабрики – ShapeCreator, создающий фигуры с интерфейсом IShape из строк. Этот класс можно снабдить интерфейсом и получить паттерн Абстрактная Фабрика. Также здесь реализуется интерфейс для Наблюдателя в виде события изменения процента выполненной работы на один процент, на который подписываются делегаты в наблюдателях (в данном случае это форма, наблюдатель используется для отображения прогресс-бара в форме).

Диаграмма классов, связанных с Презентером, показана на Рисунке 2. Тут выделен Presenter, который управляет видами через интерфейс IView.

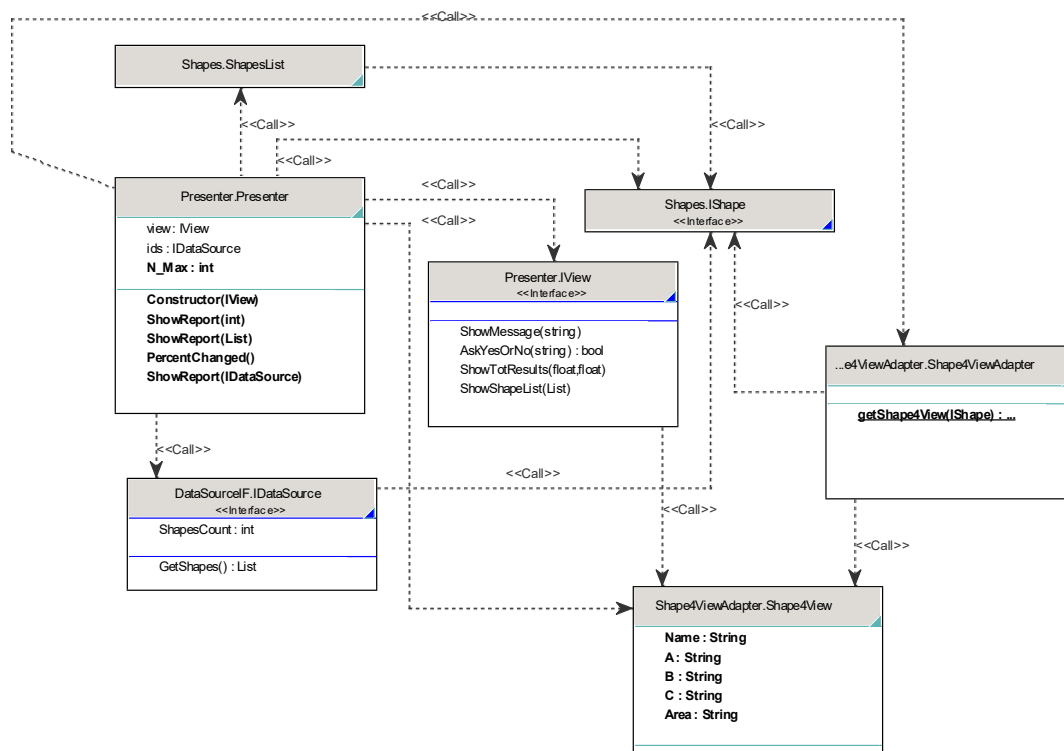


Рисунок 2 – Презентер и связанные с ним классы

Презентер управляет видом, также он работает с источниками данных `IDataSource`. При этом он может использовать любой `IDataSource`, а также знает о конкретном `IDataSource`, реализованном в программе (константный).

Помимо этого Презентер использует Адаптер `Shape4ViewAdapter` для конвертирования фигур в представление `Shape4View`, в котором нуждается веб-представление.

3 Создание и тестирование RESTful веб-сервиса на базе разработанных классов и размещение его в облаке

Рассмотрим создание веб-функции с REST API на базе рассмотренных выше классов в Azure из Visual Studio 2017 [3-4].

В данном случае не требуется дополнительно ничего устанавливать, если при установке Visual Studio Вы указали инструменты Azure.

Создаем проект функции Azure (рисунок 3) и далее выбираем `Http Trigger`.

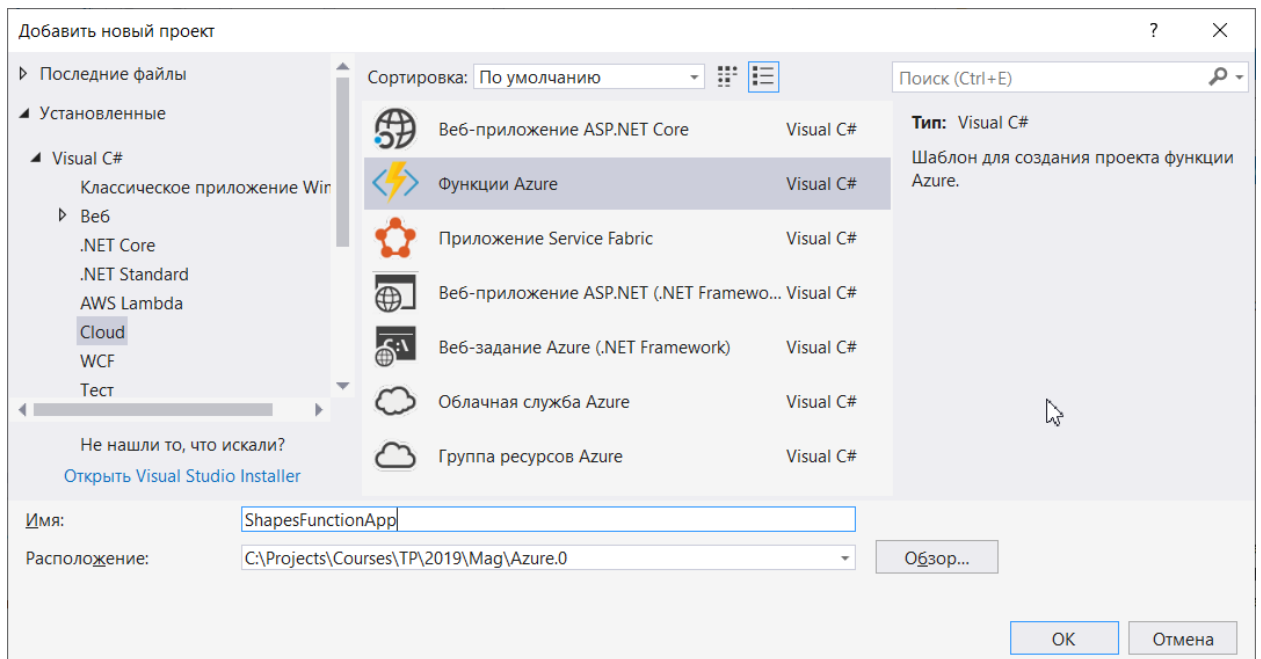


Рисунок 3 – Создание проекта функции Azure

Затем создаем код самой функции, в целом аналогичный рассмотренному выше для AWS.

Также задействуем ранее созданный класс LogView, реализующий рассмотренный ранее IView, а код самой веб-функции будет следующим (используется запрос GET):

```
public static class ShapesFunction
{
    [FunctionName("ShapesFunction")]
    public static async Task<IActionResult> Run(
        [HttpTrigger(AuthorizationLevel.Function, "get", "post",
            Route = null)] HttpRequest req,
        ILogger log)
    {
        string input = req.Query["ID"];
        LogView lv = new LogView();
        Presenter.Presenter pres = new Presenter.Presenter(lv);
        int id = Int32.Parse(input);
        if (id != 3)
        {
            pres.ShowReport(id);
        }
        else
        {

```

```

        List<string> lst = new List<string>();
        lst.Add(req.Query["shape"] + ";" +
                req.Query["a"] + ";" +
                req.Query["b"] + ";" +
                req.Query["c"]);
        pres.ShowReport(lst);
    }

    string requestBody = await new
        StreamReader(req.Body).ReadToEndAsync();
    dynamic data = JsonConvert.DeserializeObject(requestBody);

    return (ActionResult)new OkObjectResult(lv.Log);
}
}

```

После компиляции и сборки проекта, если веб-функция установлена как проект по умолчанию, ее можно запустить на локальном компьютере во встроенном веб-сервере и протестировать через браузер, о чем сообщает после запуска текст в консоли (рисунок 4), отладка поддерживается :

```

C:\Users\Andrei\AppData\Local\AzureFunctionsTools\Releases\2.43.0\cli\func.exe
[15.01.2020 19:35:26] Starting JobHost
[15.01.2020 19:35:26] Starting Host (HostId=desktope77tglc-2114454069, InstanceId=aad9a3d0-50a4-4121-87f7-38971d49fed2,
Version=2.0.12888.0, ProcessId=6540, AppDomainId=1, InDebugMode=False, InDiagnosticMode=False, FunctionsExtensionVersion
=(null))
[15.01.2020 19:35:26] Loading functions metadata
[15.01.2020 19:35:26] 1 functions loaded
[15.01.2020 19:35:26] Generating 1 job function(s)
[15.01.2020 19:35:26] Found the following functions:
[15.01.2020 19:35:26] ShapesFunctionApp.Function1.Run
[15.01.2020 19:35:26]
[15.01.2020 19:35:26] Initializing function HTTP routes
[15.01.2020 19:35:26] Mapped function route 'api/Function1' [get,post] to 'Function1'
[15.01.2020 19:35:26]
[15.01.2020 19:35:26] Host initialized (548ms)
[15.01.2020 19:35:26] Host started (560ms)
[15.01.2020 19:35:26] Job host started
Hosting environment: Production
Content root path: C:\Projects\Courses\TP\2019\Mag\Azure.0\ShapesFunctionApp\bin\Debug\netcoreapp2.1
Now listening on: http://0.0.0.0:7071
Application started. Press Ctrl+C to shut down.

Http Functions:

    Function1: [GET,POST] http://localhost:7071/api/Function1

[15.01.2020 19:35:32] Host lock lease acquired by instance ID '00000000000000000000000000000000117404C7'.

```

Рисунок 4 – Запуск веб-функции для тестирования на локальной машине

Протестировать функцию можно в браузере, указав выведенный выше в окне путь (рисунок 5)

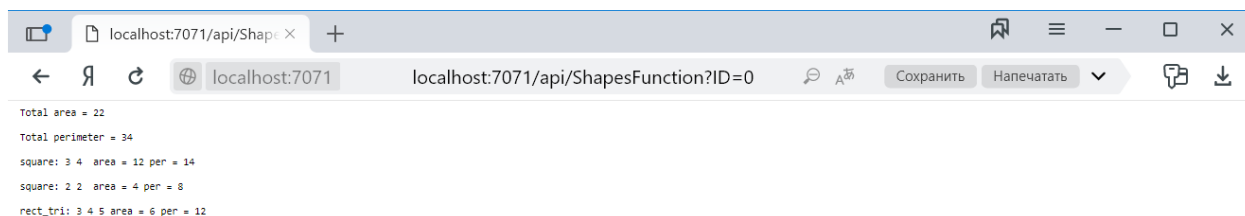


Рисунок 5 – Запуск веб-функции в браузере на локальном сервере

Далее можно создать веб-функцию в Azure, выбрав в контекстном меню проекта пункт Опубликовать (вот тут как раз потребуется аккаунт Azure !) – рисунок 6 и далее – рисунок 7.

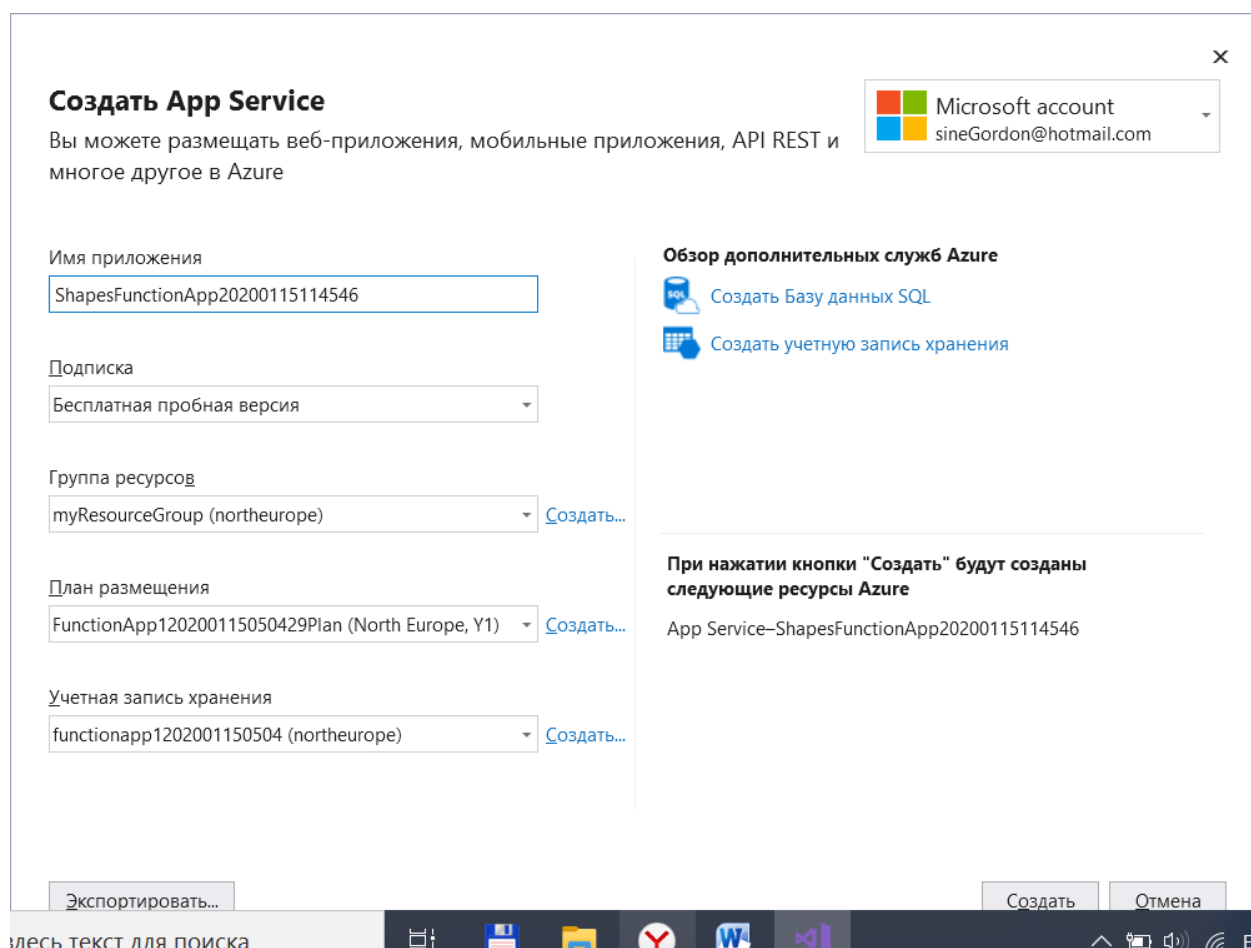


Рисунок 6 – Публикация функции в Azure

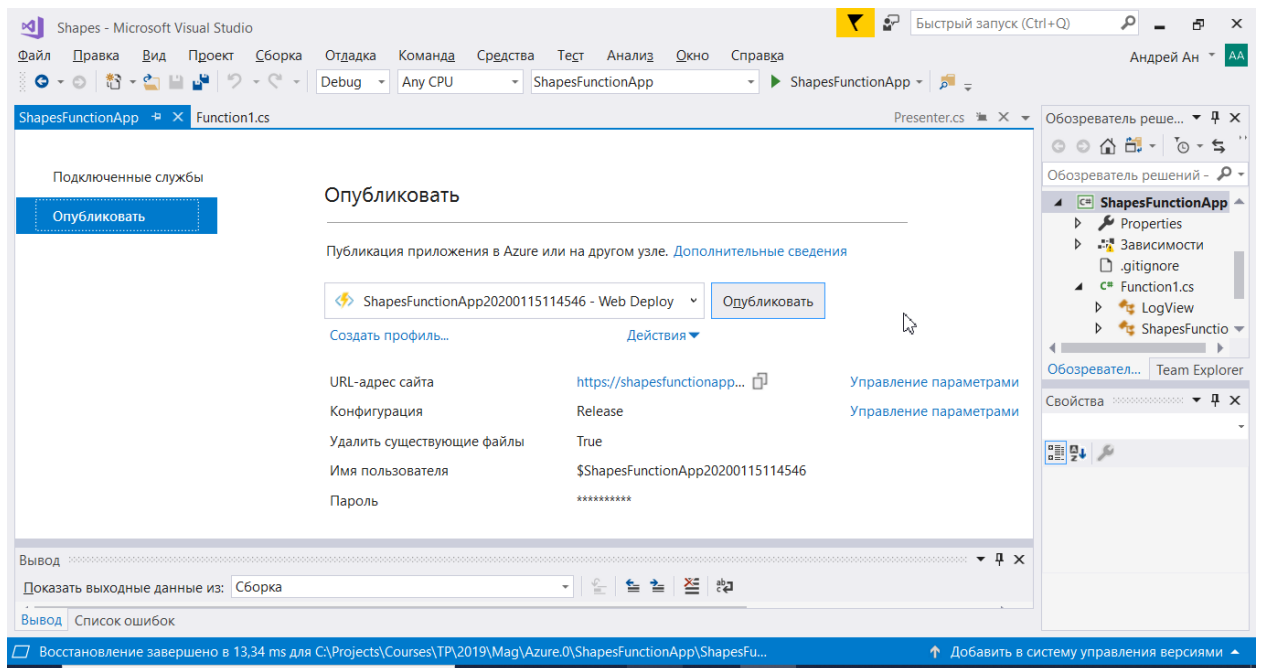


Рисунок 7 – Публикация функции в Azure (завершение)

Далее выбрав указанный на странице URL-адрес сайта и указав параметры (/api/ShapesFunction?ID=0) по идее мы должны вызвать функцию, однако при указании такого URL и параметров мы получаем ошибку, а при выборе просто указанного URL – информационную страницу о работе функции (рисунок 8) :

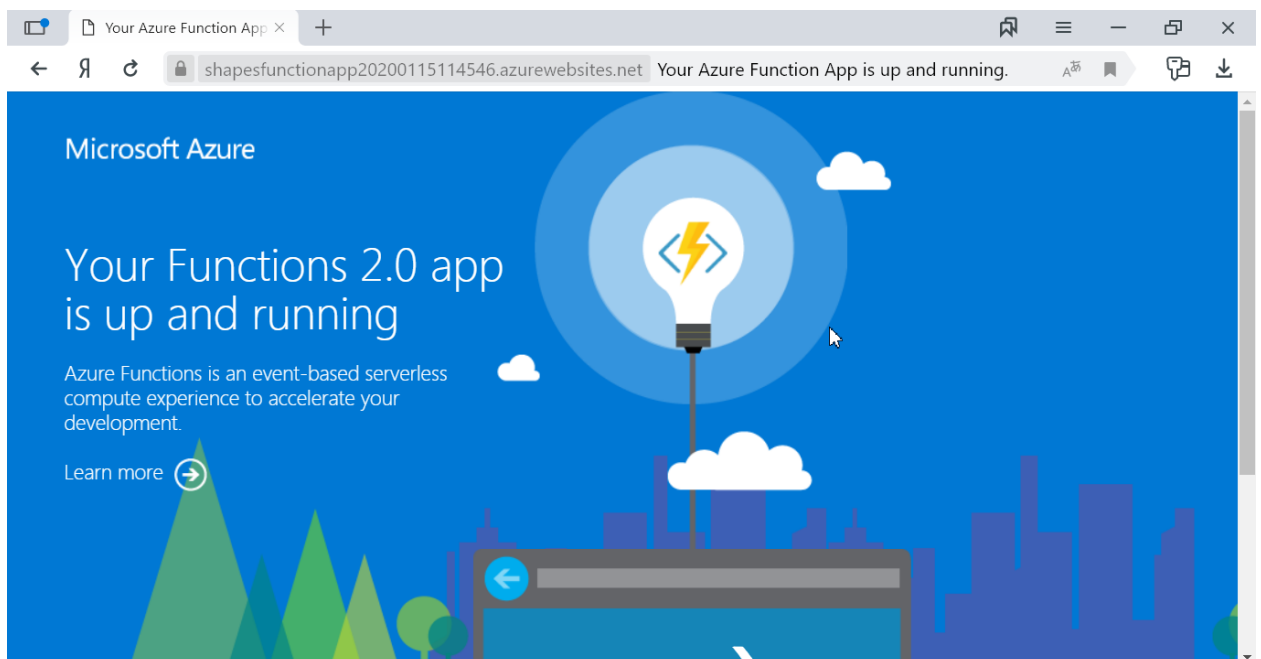


Рисунок 8 – Информация о работе приложения при выборе его URL

Подобное поведение функции не соответствует документации, возможно, это связано с использованием VS2017, а не VS2019, либо с конфигурацией VS / ПК. Для получения реальной ссылки с дополнительными параметрами мы можем зайти в консоль (портал) Azure в свой кабинет, найти функцию и там нажать на Получить URL-адрес функции (рисунок 9), скопировать и сохранить строку запроса.

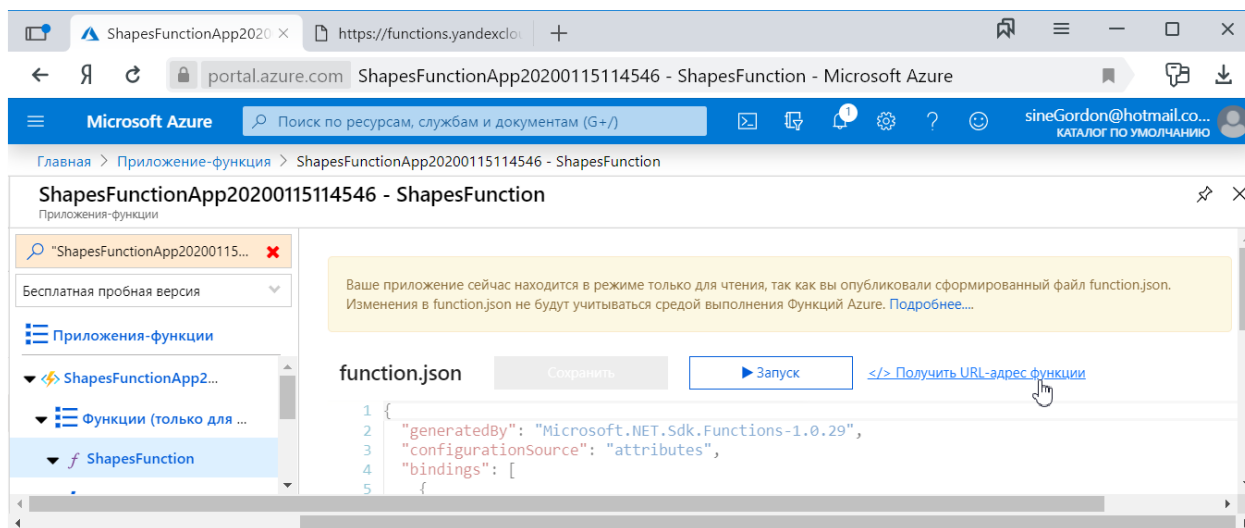


Рисунок 9 – Получение URL веб-функции на портале Azure

Для вызова нашей веб-функции нужно указать строку :

<https://shapesfunctionapp20200115114546.azurewebsites.net/api/ShapesFunction?code=8SPvE4JqNI1K9LwLgTWZsOLK51GOwBvpzPExeHNrAkHD55qEr/VWAg==&ID=0>

Ответ функции в браузере показан на рисунке 10

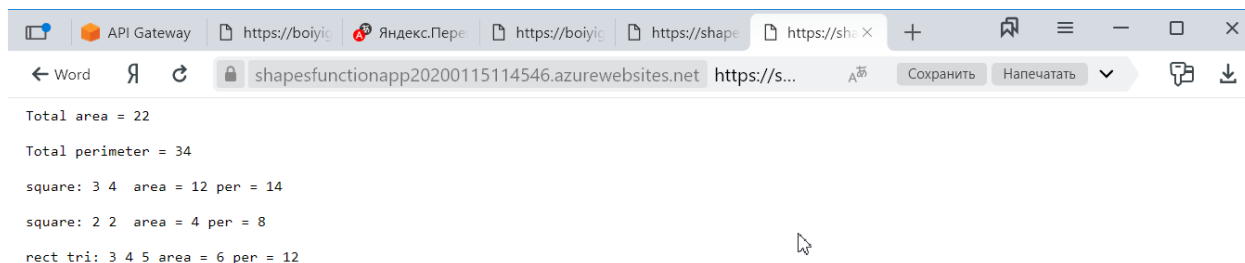


Рисунок 10 – Ответ функции в браузере (ID=0)

Заключение

Рассмотрен простой пример работы с иерархиями объектов, созданный с их помощью простой веб-сервис и его размещение в облаке Azure.

Список использованных источников :

1. Андреев, А.Е. Адаптивные технологии разработки программного обеспечения: учеб. пособие / А.Е. Андреев, С.И. Кириносенко; ВолгГТУ. - Волгоград, 2015. - 95 с.
2. Паттерны проектирования в C# и .NET [Электронный ресурс] Режим доступа : <https://metanit.com/sharp/patterns/> (дата обращения : 25.12.2020)
3. Документация по функциям Azure [Электронный ресурс] Режим доступа : <https://docs.microsoft.com/ru-ru/azure/azure-functions/> (дата обращения : 25.12.2020)
4. Создание первой функции в Azure с помощью Visual Studio [Электронный ресурс] Режим доступа : <https://docs.microsoft.com/ru-ru/azure/azure-functions/functions-create-your-first-function-visual-studio> (дата обращения : 25.12.2020)